

Exit Codes

- 0 – normal execution
- 1 – for any error

See Also

csplit(C), fspec(C), tabs(C)

Diagnostics

All diagnostics are fatal.

usage: . . .

Newform was called with a bad option.

not -s format

There was no TAB on one line.

can't open file

Self-explanatory.

internal line too long

A line exceeds 512 characters after being expanded in the internal work buffer.

tabspec in error

A tab specification is incorrectly formatted, or specified tab stops are not ascending.

tabspec indirection illegal

A *tabspec* read from a file (or standard input) may not contain a *tabspec* referencing another file (or standard input).

Notes

Newform normally only keeps track of physical characters; however, for the *-i* and *-o* options, *newform* will keep track of BACKSPACES in order to line up TABs in the appropriate logical columns.

Newform will not prompt the user if a *tabspec* is to be read from the standard input (by use of *-i*— or *-o*—).

If the *-f* option is used, and the last *-o* option specified was “*-o*—”, and was preceded by either “*-o*—” or “*-i*—”, the tab specification format line will be incorrect.



Name

`newgrp` – Logs user in to a new group.



Syntax

`newgrp [ group ]`

Description

Newgrp changes the group identification of its caller. The same person remains logged in, and the current directory is unchanged, but calculations of access permissions to files are performed with respect to the new group ID.

Newgrp without an argument changes the group identification to the group in the password file; in effect, it changes the group identification back to the caller's original group.

When most users log in, they are members of the group named `group`.

Files



`/etc/group`

`/etc/passwd`

Notes

If the *newgrp* command fails, it still execs a new shell that does not retain the characteristics of the previous shell.

Diagnostics

Sorry

Group change request failed because the user is not listed as a member of the group requested. The list of users for each group is in `/etc/group` and can be edited by the super-user to include more members.



See Also

`group(M)`, `login(M)`



Name

`news` – Print news items.

Syntax

`news [ -a | -n | -s | items ]`

Description

News is used to keep the user informed of current events. By convention, these events are described by files in the `/usr/news` directory.

When invoked without arguments, *news* prints the contents of all current files in `/usr/news`, most recent first, with each preceded by an appropriate header. *News* stores the “currency” time as the modification date of a `.news_time` file in your home directory (the identity of this directory is determined by the environment variable, `$HOME`); only files more recent than this current time are considered “current.”

The `-a` option causes *news* to print all items, regardless of currency. In this case, the stored time is not changed.

The `-n` option causes *news* to report the names of the current items without printing their contents, and without changing the stored time.

The `-s` option causes *news* to report how many current items exist, without printing their names or contents, and without changing the stored time.

All other arguments are assumed to be specific news items that are to be printed.

If the INTERRUPT key is struck during the printing of a news item, printing stops and the next item is started. Another INTERRUPT within one second of the first causes the program to terminate.

Files

`/usr/news/*`
`$HOME/.news_time`

See Also

`enviro.(M)`, `profile.(M)`



Name

`nice` – Runs a command at a different priority.

Syntax

`nice [ -increment ] command [ arguments ]`

Description

Nice executes *command* with a lower CPU scheduling priority. Priorities range from 0 to 39, where 0 is the highest priority and 39 is the lowest. By default, commands have a priority of 20. If an *-increment* argument is given, where *increment* is in the range 1-19, *increment* is added to the default priority of 20 to produce a numerically higher priority, meaning a *lower* scheduling priority. If no *increment* is given, an increment of 10 to produce a priority of 30 is assumed.

The super-user may run commands with a *higher* priority than normal by using a double negative *increment*. For example, an argument of “--10” would decrement the default to produce a priority of 10, which is a higher scheduling priority than the default of 20.

See Also

`nice(S)`, `nohup(C)`

Diagnostics

Nice returns the exit status of the subject command.

Notes

An *increment* larger than 19 is equivalent to 19.



Name

r1 – Adds line numbers to a file.

Syntax

nl [-h *type*] [-b *type*] [-f *type*] [-v *start* #] [-i *incr*] [-p] [-l *num*] [-s *sep*] [-w *width*] [-n *format*] [*file*]

Description

Nl reads lines from the named *file*, or from the standard input if no *file* is named, and reproduces the lines on the standard output. Lines are numbered on the left in accordance with the command options in effect.

Nl views the text it reads in terms of logical pages. Line numbering is reset at the start of each logical page. A logical page consists of a header, a body, and a footer section. Empty sections are valid. Different line numbering options are available independently for header, body, and footer (e.g. no numbering of header and footer lines while numbering blank lines in the body).

The start of logical page sections is signaled by input lines containing nothing but the following character(s):

Page Section	Line Contents
Header	\:\:
Body	\:
Footer	\:

Unless signaled otherwise, *nl* assumes the text being read is in a single logical page body.

Command options may appear in any order and may be intermingled with an optional filename. Only one file may be named. The options are:

- h*type*** Same as **-b*type*** except for header. The default *type* for logical page header is **n** (no lines numbered).
- b*type*** Specifies which logical page body lines are to be numbered. Recognized *types* and their meaning are: **a**, number all lines; **t**, number lines with printable text only; **n**, no line numbering; **pstring**, number only lines that contain the regular expression specified in *string*. The default *type* for logical page body is **t** (text lines numbered).

- ftype** Same as **-btype** except for footer. The default for logical page footer is **n** (no lines numbered).
- p** Does not restart numbering at logical page delimiters.
- vstart#** *Start#* is the initial value used to number logical page lines. The default is **1**.
- iincr** *Incr* is the increment value used to number logical page lines. The default is **1**.
- ssep** *Sep* is the character(s) used in separating the line number and the corresponding text line. You must enclose *sep* within quotes. Default *sep* is a TAB.
- wwidth** *Width* is the number of characters to be used for the line number. The default *width* is **6**.
- nformat** *Format* is the line numbering format. The recognized values are: **ln**, left justified, leading zeroes suppressed; **rn**, right justified, leading zeroes suppressed; **rz**, right justified, leading zeroes kept. The default *format* is **rn** (right justified).
- lnum** *Num* is the number of blank lines to be considered as one. For example, **-12** results in only the second adjacent blank being numbered (if the appropriate **-ha**, **-ba**, and/or **-fa** option is set). The default is **1**.

See Also

pr(C)

Name

nohup – Runs a command immune to hangups and quits.

Syntax

nohup *command* [*arguments*]

Description

Nohup executes *command* with hangups and quits ignored. If output is not redirected by the user, it will be sent to **nohup.out**. If **nohup.out** is not writable in the current directory, output is redirected to **\$HOME/nohup.out**.

See Also

nice(C), **signal**(S)



Name

`od` – Displays files in octal format.

Syntax

`od [-bcdx] [file] [ [+ ] offset: [ . ] [ b ] ]`

Description

Od displays *file* in one or more formats as selected by the first argument. If the first argument is missing, `-o` is default. The meanings of the format options are:

- `-b` Interprets bytes in octal.
- `-c` Interprets bytes in ASCII. Certain nongraphic characters appear as C escapes: `null=\0`, `BACKSPACE=\b`, `FORMFEED=\f`, `NEWLINE=\n`, `RETURN=\r`, `TAB=\t`; others appear as 3-digit octal numbers.
- `-d` Interprets words in decimal.
- `-o` Interprets words in octal.
- `-x` Interprets words in hexadecimal.

The *file* argument specifies which file is to be displayed. If no *file* argument is specified, the standard input is used.

The offset argument specifies the offset in the file where displaying is to start. This argument is normally interpreted as octal bytes. If `.` is appended, the offset is interpreted in decimal. If `b` is appended, the offset is interpreted in blocks of `BSIZE` as defined in `/usr/include/sys/param.h`. If the file argument is omitted, the offset argument must be preceded by `+`.

The display continues until end-of-file.

See Also

`adb(CP)`, `hd(C)`



Name

pack, pcat, unpack – Compresses and expands files.

Syntax

pack [-] *name* ...

pcat *name* ...

unpack *name* ...

Description

Pack attempts to store the specified files in a compressed form. Wherever possible (and useful), each input file *name* is replaced by a packed file *name.z* with the same access modes, access and modified dates, and owner as those of *name*. If *pack* is successful, *name* will be removed. Packed files can be restored to their original form using *unpack* or *pcat*.

Pack uses Huffman (minimum redundancy) codes on a byte-by-byte basis. If the - argument is used, an internal flag is set that causes the number of times each byte is used, its relative frequency, and the code for the byte to be printed on the standard output. Additional occurrences of - in place of *name* will cause the internal flag to be set and reset.

The amount of compression obtained depends on the size of the input file and the character frequency distribution. Because a decoding tree forms the first part of each .z file, it is usually not worthwhile to pack files smaller than three blocks, unless the character frequency distribution is very skewed, which may occur with printer plots or pictures.

Typically, text files are reduced to 60-75% of their original size. Load modules, which use a larger character set and have a more uniform distribution of characters, show little compression, the packed versions being about 90% of the original size.

Pack returns a value that is the number of files that it failed to compress.

No packing will occur if:

- The file appears to be already packed
- The filename has more than 12 characters
- The file has links

- The file is a directory
- The file cannot be opened
- No disk storage blocks will be saved by packing
- A file called *name.z* already exists
- The *.z* file cannot be created
- An I/O error occurred during processing

The last segment of the filename must not contain more than 12 characters to allow space for the appended *.z* extension. Directories cannot be compressed.

Pcat does for packed files what *cat*(C) does for ordinary files. The specified files are unpacked and written to the standard output. Thus, to view a packed file named *name.z* use:

```
pcat name.z
```

or just:

```
pcat name
```

To make an unpacked copy, say *nnn*, of a packed file, *name.z* (without destroying *name.z*), use the command:

```
pcat name >nnn
```

Pcat returns the number of files it was unable to unpack. Failure may occur if:

- The filename (exclusive of the *.z*) has more than 12 characters
- The file cannot be opened
- The file does not appear to be the output of *pack*

Unpack expands files created by *pack*. For each *name* file specified in the command, a search is made for a file called *name.z* (or just *name*, if *name* ends in *.z*). If this file appears to be a packed file, it is replaced by its expanded version. The new file has the *.z* suffix stripped from its name, and has the same access modes, access and modification dates, and owner as those of the packed file.

Unpack returns a value that is the number of files it was unable to unpack. Failure may occur for the same reasons that it may in *pcat*, as well as in a file where the “unpacked” name already exists, or if the unpacked file cannot be created.

PACK(C)

PACK(C)

See Also

cat(C)



Name

`passwd` – Changes login password.

Syntax

`passwd name`

Description

This command changes (or installs) a password associated with the login *name*.

The program prompts for the old password (if any) and then for the new one (twice). The user must supply these. Passwords can be of any reasonable length, but only the first eight characters of the password are significant. The minimum number of characters allowed in a new password is determined by the `PASSLENGTH` variable. Although the minimum can be three, a minimum of five characters is strongly recommended since passwords shorter than this are much easier to guess or discover by trial and error.

Only the owner of the name or the super-user may change a password; the owner must prove he knows the old password. Only the super-user can create a null password.

The password file is not changed if the new password is the same as the old password, or if the password has not “aged” sufficiently; see *passwd(M)*.

The minimum length of a legal password, and the minimum and maximum number of weeks used in password aging are specified in `/etc/default/passwd` by the variables `PASSLENGTH`, `MINWEEKS`, and `MAXWEEKS`. If not explicitly set, the default values for these variables are:

```
PASSLENGTH=5
MINWEEKS=2
MAXWEEKS=4
```

`MINWEEKS` and `MAXWEEKS` values must be in the range 0 to 63. If `PASSLENGTH` is not in the range 3 to 8, it is set to 5.

Files

`/etc/default/passwd`
`/etc/passwd`

See Also

`default(M)`, `login(M)`, `passwd(M)`, `pwadmin(C)`



Name

pg – File perusal filter for soft-copy terminals.

Syntax

```
pg [- number] [-p string] [-cefns] [+ linenumber] [+ / pattern /] [files ...]
```

Description

The *pg* command is a filter which allows the examination of *files* one screenful at a time on a soft-copy terminal. (The dash (–) command line option and/or null arguments indicate that *pg* should read from the standard input.) Each screenful is followed by a prompt. If you press the RETURN key, another page is displayed; other possibilities are listed below. This command is different from previous paginators because it allows you to back up and review something that has already passed.

To determine terminal attributes, *pg* scans the *termcap*(M) database for the terminal type specified by the environment variable TERM. If TERM is not defined, the terminal type, *dumb*, is assumed.

The command line options are:

- number* Specifies the size (in lines) of the window that *pg* is to use instead of the default. (On a terminal containing 24 lines, the default window size is 23.)
- p string* Causes *pg* to use *string* as the prompt. If the prompt string contains a “%d”, the first occurrence of “%d” in the prompt will be replaced by the current page number when the prompt is issued. The default prompt string is a colon (:).
- c* Moves the cursor home and clears the screen before displaying each page. This option is ignored if *clear_screen* is not defined for this terminal type in the *termcap*(M) database.
- e* Causes *pg* *not* to pause at the end of each file.
- f* Inhibits *pg* from splitting lines. In the absence of the *-f* option, *pg* splits lines longer than the screen width, but some sequences of characters in the displayed text (for example, escape sequences for underlining) give undesirable results.
- n* Normally, commands must be terminated by pressing the RETURN key (ASCII NEWLINE character). This option causes an automatic end of command as soon as a command letter is entered.

- s Causes *pg* to print all messages and prompts in standout mode (usually inverse video).
- +*linenumber* Starts up at *linenumber*.
- +/*pattern*/ Starts up at the first line containing the regular expression pattern.

The responses that may be typed when *pg* pauses can be divided into three categories: those that allow further perusal, those that search, and those that modify the perusal environment.

Commands which allow further perusal normally take a preceding *address*, an optionally signed number indicating the point from which further text should be displayed. *Pg* interprets this *address* in either pages or lines depending on the command. A signed *address* specifies a point relative to the current page or line, and an unsigned *address* specifies an address relative to the beginning of the file. Each command has a default address of +1.

The perusal commands and their defaults are as follows:

- [+|_|n]RETURN key or <blank> Causes one page to be displayed. The *address* is specified in pages.
- [+|_|n]l With a signed *address*, causes *pg* to simulate scrolling the screen, forward or backward, the number of lines specified. With an unsigned *address* this command prints a full screen beginning at the specified line.
- [+|_|d or CTRL-D Simulates scrolling half a screen forward or backward. Scrolling is always relative to the current line.

The following perusal commands take no *address*:

- . or CTRL-L Causes the current page of text to be redisplayed.
- \$ Displays the last windowful in the file. Use with caution when the input is a pipe.

The following commands are available for searching for text patterns in the text. The regular expressions described in *ed*(C) are available. They must always be terminated by a NEWLINE character, even if the —n option is specified.

- i*/*pattern*/ Search forward for the *i*th (default *i*=1) occurrence of *pattern*. Searching begins immediately after the current page and continues to the end of the current file, without wraparound.

*i?pattern^**i?pattern?*

Search backwards for the *i*th (default *i*=1) occurrence of *pattern*. Searching begins immediately before the current page and continues to the beginning of the current file, without wraparound. The caret (^) notation is useful for terminals which will not properly handle the question mark (?).

After searching, *pg* displays the line found at the top of the screen. You can modify this by appending *m* or *b* to the search command to leave the line found in the middle or at the bottom of the window from now on. Use the *t* suffix to restore the original situation.

You can modify the environment of perusal with the following commands:

in Begins perusing the *i*th next file in the command line. The *i* is an unsigned number, whose default value is 1.

ip Begins perusing the *i*th previous file in the command line. The *i* is an unsigned number, whose default is 1.

iw Displays another window of text. If *i* is present, set the window size to *i*.

sfilename Saves the input in the named file. Only the current file being perused is saved. The white space between the *s* and *filename* is optional. This command must always be terminated by a NEWLINE character, even if the **-n** option is specified.

h Help by displaying an abbreviated summary of available commands.

q or **Q** Quit *pg*.

!command *Command* is passed to the shell, whose name is taken from the SHELL environment variable. If this is not available, the default shell is used. This command must always be terminated by a NEWLINE character, even if the **-n** option is specified.

At any time when output is being sent to the terminal, the user can press the QUIT key (normally or the INTERRUPT (BREAK) key). This causes *pg* to stop sending output, and display the prompt. The user may then enter one of the above commands in the normal manner. Unfortunately, some output is lost when this is done, because any characters waiting in the terminal's output queue are flushed when the quit signal occurs.

If the standard output is not a terminal, then *pg* acts just like *cat*(C), except that a header is printed before each file (if there is more than one).

Example

To use *pg* to read system news, type:

```
news | pg -p "(Page %d):"
```

Files

/etc/termcap	Terminal information database
/tmp/pg*	Temporary file when input is from a pipe

See Also

ed(C), grep(C), termcap(M)

Notes

Undesirable results may occur if terminal tabs are not set every eight positions,

When using *pg* as a filter with another command that changes the terminal I/O options, terminal settings may not be restored correctly.

While waiting for terminal input, *pg* responds to BREAK, DEL, and the caret (^) by terminating execution. Between prompts, however, these signals interrupt *pg*'s current task and place you in prompt mode. Use these signals with caution when input is being read from a pipe, since an interrupt is likely to terminate the other commands in the pipeline.

The *z* and *f* commands used with *more* are available, and the terminal slash (/), caret (^), or question mark (?) may be omitted from the searching commands.

Name

pr – Prints files on the standard output.

Syntax

pr [*options*] [*files*]

Description

Pr prints the named files on the standard output. If *file* is **–**, or if no files are specified, the standard input is assumed. By default, the listing is separated into pages, each headed by the page number, date and time, and the name of the file.

By default, columns are of equal width, separated by at least one space; lines which do not fit are truncated. If the **–s** option is used, lines are not truncated and columns are separated by the specified separation character (see **–s**).

If the standard output is associated with a terminal, error messages are withheld until *pr* has completed printing.

Options may appear singly or be combined in any order. Their meanings are:

- +k** Begins printing with page *k* (default is 1).
- k** Produces *k*-column output (default is 1). The **–e** and **–i** options are assumed for multi-column output.
- a** Prints multi-column output across the page.
- m** Merges and prints all files simultaneously, one per column (overrides the **–k**, and **–a** options).
- d** Double-spaces the output.
- eck** Expands *input* TABs to character positions $k+1$, $2*k+1$, $3*k+1$, etc. If *k* is 0 or is omitted, default tab settings at every 8th position are assumed. TAB characters in the input are expanded into the appropriate number of spaces. If *c* (any nondigit character) is given, it is treated as the input TAB character (the default for *c* is the TAB character).
- ick** In *output*, replaces white space wherever possible by inserting TABs to character positions $k+1$, $2*k+1$, $3*k+1$, etc. If *k* is 0 or is omitted, default tab settings at every 8th position are assumed. If *c* (any nondigit character) is given, it is treated as the output TAB character (The default for *c* is the TAB character.).

- nck* Provides *k*-digit line numbering (default for *k* is 5). The number occupies the first *k*+1 character positions of each column of normal output or each line of –*m* output. If *c* (any nondigit character) is given, it is appended to the line number to separate it from whatever follows (the default for *c* is a TAB).
- wk* For multi-column output only, this option sets the page width to *n* characters instead of the default width of 72 characters.
- ok* Offsets each line by *k* character positions (the default offset is 0.). The number of character positions per line is the sum of the width and offset.
- lk* Sets the length of a page to *k* lines (the default length is 66.). The number of lines printed on each page is equal to *k* minus 10. If *k* is less than or equal to 10, page headings are disabled.
- h* Uses the next argument as the header to be printed instead of the filename.
- p* Pauses before beginning each page if the output is directed to a terminal (*pr* will ring the bell at the terminal and wait for a RETURN).
- f* Uses FORMFEED character for new pages (the default is to use a sequence of LINEFEEDs.). Pauses before beginning the first page if the standard output is associated with a terminal.
- r* Prints no diagnostic reports on failure to open files.
- t* Prints neither the 5-line identifying header nor the 5-line trailer normally supplied for each page. Quits printing after the last line of each file without spacing to the end of the page.
- sc* Separates columns by the single *c* character instead of by the appropriate number of spaces (the default for *c* is a TAB).

Examples

The following prints *file1* and *file2* as a double-spaced, three-column listing headed by “file list”:

```
pr -3dh "file list" file1 file2
```

The following writes *file1* on *file2*, expanding tabs to columns 10, 19, 28, 37, . . . :

```
pr -e9 -t <file1 >file2
```

See Also

cat(C)

Name

ps – Reports process status.

Syntax

ps [*options*]

Description

Ps prints certain information about active processes. Without *options*, information is printed about processes associated with the current terminal. Otherwise, displayed information is controlled by the following *options*:

- e Prints information about all processes.
- d Prints information about all processes, except process group leaders.
- a Prints information about all processes, except process group leaders and processes not associated with a terminal.
- f Generates a *full* listing. (Normally, a short listing containing only process ID, terminal (“tty”) identifier, cumulative execution time, and the command name is printed.) See below for meaning of columns in a full listing.
- l Generates a *long* listing. See below.
- c *corefile* Uses the file, *corefile*, in place of /dev/mem.
- s *swapdev* Uses the file, *swapdev*, in place of /dev/swap. This is useful when examining a *corefile*.
- n *namelist* The argument is taken as the name of an alternate *namelist* (/xenix is the default).
- k *kmemstart* Substitutes the value of *kmemstart* for the value specified in the header of the *namelist* file.
- t *tlist* Restricts listing to data about the processes associated with the terminals given in *tlist*, where *tlist* can be in one of two forms: a list of terminal identifiers separated from one another by a comma, or a list of terminal identifiers enclosed in double quotes and separated from one another by a comma and one or more spaces.

- p *plist*** Restricts listing to data about processes whose process ID numbers are given in *plist*, where *plist* is in the same format as *tlist*.
- u *ulist*** Restricts listing to data about processes whose user ID numbers or login names are given in *ulist*, where *ulist* is in the same format as *tlist*. In the listing, the numerical user ID is printed unless the **-f** option is used, in which case the login name is printed.
- g *glist*** Restricts listing to data about processes whose process groups are given in *glist*, where *glist* is a list of process group leaders and is in the same format as *tlist*.

The column headings and the meaning of the columns in a *ps* listing are given below; the letters *f* and *l* indicate the option (*full* or *long*) that causes the corresponding heading to appear; *all* means that the heading always appears. Note that these two options only determine what information is provided for a process; they do *not* determine which processes will be listed.

- F** (l) A status word consisting of flags associated with the process. Each flag is associated with a bit in the status word. These flags are added to form a single octal number. Process flag bits and their meanings are:

01	in core
02	system process
04	locked in core (e.g., for physical I/O)
10	being swapped
20	being traced by another process

- S** (l) The state of the process:

0	non-existent
S	sleeping
W	waiting
R	running
I	intermediate
Z	terminated
T	stopped

- UID** (f,l) The user ID number of the process owner; the login name is printed under the **-f** option.

- PID** (all) The ID of the process; it is possible to kill a process if you know this datum.

- PPID** (f,l) The ID of the parent process.

- C** (f,l) Processor utilization for scheduling.

STIME	(f)	Starting time of the process.
PRI	(l)	The priority of the process; higher numbers mean lower priority.
NI	(l)	Nice value; used in priority computation.
ADDR	(l)	The memory address of the process, if resident; otherwise, the disk address.
SZ	(l)	The size in blocks of the core image of the process, but not including the size of text shared with other processes. Since this size includes the current size of the stack, it will vary as the stack size varies. The number of bytes in a block is BSIZE as defined in <code>/usr/include/sys/param.h</code> .
WCHAN	(l)	The event for which the process is waiting or sleeping; if blank, the process is running.
TTY	(all)	The controlling terminal for the process.
TIME	(all)	The cumulative execution time for the process.
CMD	(all)	The command name; the full command name and its arguments are printed under the <code>-f</code> option.

A process that has exited and has a parent, but has not yet been waited for by the parent, is marked `<defunct>`.

Under the `-f` option, *ps* tries to determine the command name and arguments given when the process was created by examining memory or the swap area. Failing this, the command name, as it would appear without the `-f` option, is printed in square brackets.

Files

<code>/xenix</code>	System namelist
<code>/dev/mem</code>	Memory
<code>/dev</code>	Searched to find swap device and terminal ("tty") names.

See Also

`kill(C)`, `nice(C)`

Notes

Things can change while *ps* is running; the picture it gives is only a close approximation to reality.

Some data printed for defunct processes are irrelevant.

The *ps* command with the *-f* option may not display the correct command name for a process which

Name

pstat – Reports system information.

Syntax

```
pstat [ -aixpf ] [ -u ubase ] [ -c corefile ]
[ -n namelist ] [ -k kmemstart ]
```

Description

Pstat interprets the contents of certain system tables. *Pstat* searches for these tables in */dev/mem* and */dev/kmem*. The *-c* option allows one to specify a *corefile* rather than */dev/kmem* for the search. Similarly, the *-n* option lets you specify a *namelist* rather than */xenix* and the *-k* option lets you specify a different value for *kmemstart* than the one in the *namelist* header. Options are:

-a Under **-p**, describe all process slots rather than just active ones.

-i Print the inode table with these headings:

LOC	The core location of this table entry.
FLAGS	Miscellaneous state variables encoded thus:
	L Locked.
	U Update time <i>filesystem</i> (F) must be corrected.
	A Access time must be corrected.
	M File system is mounted here.
	W Wanted by another process (L flag is on).
	T Contains a text file.
	C Changed time must be corrected.
CNT	Number of open file table entries for this inode.
DEV	Major and minor device number of file system in which this inode resides.
INO	I-number within the device.
MODE	Mode bits, see <i>chmod</i> (S).
NLK	Number of links to this inode.
UID	User ID of owner.
SIZ/DEV	Number of bytes in an ordinary file, or major and minor device of special file.

-x Prints the text table with these headings:

LOC	The core location of this table entry.
-----	--

FLAGS	Miscellaneous state variables encoded thus: T <i>ptrace</i> (S) in effect. W Text not yet written on swap device. L Loading in progress. K Locked. w Wanted (L flag is on).
DADDR	Disk address in swap, measured in multiples of BSIZE bytes.
CADDR	Core address, measured in units of memory management resolution.
SIZE	Size of text segment, measured in units of memory management resolution.
IPTR	Core location of corresponding inode.
CNT	Number of processes using this text segment.
CCNT	Number of processes in core using this text segment.

-p Prints process table for active processes with these headings:

LOC	The core location of this table entry.
S	Run state encoded thus: 0 No process 1 Waiting for some event 3 Runnable 4 Being created 5 Being terminated 6 Stopped under trace
F	Miscellaneous state variables, connected by OR: 01 Loaded. 02 The scheduler process. 04 Locked. 010 Swapped out. 020 Traced. 040 Used in tracing. 0100 Locked in by <i>lock</i> (S).
PRI	Scheduling priority, see <i>nice</i> (S).
SIGNAL	Signals received (signals 1-16 coded in bits 0-15).
UID	Real user ID.
TIM	Time resident in seconds; times over 127 coded as 127.
CPU	Weighted integral of CPU time, for scheduler.
NI	Nice level, see <i>nice</i> (S).
PGRP	Process number of root of process group (the opener of the controlling terminal).
PID	The process ID number.

PPID	The process ID of parent process.
ADDR	If in core, the physical address of the “u-area” of the process measured in units of memory management resolution. If swapped out, the position in the swap area measured in multiples of BSIZE bytes.
SIZE	Size of process image, measured in units of memory management resolution.
WCHAN	Wait channel number of a waiting process.
LINK	Link pointer in list of runnable processes.
TEXTP	If text is pure, pointer to location of text table entry.
CLKT	Countdown for <i>alarm</i> (S) measured in seconds.

-u ubase

Print information about a user process. *Ubase* is the hexadecimal location of the process in main memory. The address may be obtained by using the long listing (**-l** option) of the *ps*(C) command.

-c corefile

Use the *corefile* file in place of */dev/kmem*.

-n namelist

Use the *namelist* file as an alternate namelist in place of */xenix*.

-k kmemstart

Substitutes the value of *kmemstart* for the value specified in the header of the *namelist* file.

-f Print the open file table with these headings:

LOC	The core location of this table entry.
FLG	Miscellaneous state variables encoded thus: R Open for reading. W Open for writing. P Pipe
CNT	Number of processes that know this open file.
INO	The location of the inode table entry for this file.
OFFS	The file offset, see <i>lseek</i> (S).

-k kmemstart

Substitutes the value of *kmemstart* for the value specified in the header of the *namelist* file.

Files*/xenix**Namelist*

/dev/mem

Default source of tables

See Also

filesystem(F), ps(C), stat(S)

Name

pwadmin – Performs password aging administration.

Syntax

pwadmin *option user*

Description

Pwadmin is used to examine and modify the password aging information in the password file. You can specify only one option on the command line with the exception of **–min** and **–max** which you can specify together on the command line. *User* is always required. The options one can specify are as follows:

- d** Displays the password aging information.
- f** Forces the user to change his password at the next login.
- c** Prevents the user from changing his password.
- a** Enables password aging for the given user. This option sets the minimum number of weeks that the user must wait before changing his password and the maximum number of weeks that a user can keep his current password to the values defined by the MINWEEKS and MAXWEEKS variables in the */etc/default/passwd* file. If the file is not found or the defined values are not in the range 0 to 63, the default values 2 and 4 are used.
- n** Disables the password aging feature.
- min *weeks***
Enables password aging and sets the minimum number of weeks before the user can change his password to *weeks*. (This prevents him from changing his password back to the old one).
- max *weeks***
Enables password aging and sets the number of weeks that the user can keep his current password to *weeks*.

Files

/etc/passwd

See Also

passwd(C), passwd(M)

Notes

The user must not attempt to force a new password by setting both the **–min** and **–max** values to zero. To force a password, use the **–f** option.

The user must not attempt to prevent further password changes by setting the **–min** value greater than the **–max** value. Use the **–c** option to prevent changes.



Name

pwcheck, pwck – Checks password file.

Syntax

pwcheck [*file*]

pwck [*file*]

Description

Pwcheck scans the password file and checks for any inconsistencies. *Pwcheck* and *pwck* are equivalent commands and may be used interchangeably. The checks include validation of the number of fields, login name, user ID, group ID, and whether the login directory and optional program name exist. The default password file is */etc/passwd*.

Files

/etc/passwd

See Also

group(M), grpcheck(C), passwd(M)



Name

pwd – Prints working directory name.

Syntax

pwd

Description

Pwd prints the pathname of the working (current) directory.

See Also

cd(C)

Diagnostics

“Can not open ..” and “Read error in ..” indicate possible file system trouble. In such cases, see the *XENIX Operations Guide* for information on fixing the file system.



Name

quot – Summarizes file system ownership.

Syntax

```
quot [ option ] ... [ filesystem ]
```

Description

Quot prints the number of blocks (512 bytes each) in the named *filesystem* currently owned by each user. If no *filesystem* is named, the file systems given in */etc/mnttab* are examined.

The following options are available:

-c

Prints three columns giving file size in blocks, number of files of that size, and cumulative total of blocks in that size or smaller file.

-f

Prints count of number of files as well as space owned by each user.

/etc/passwd

Gets user names

/etc/mnttab

Contains list of mounted file systems

See Also

du(C), ls(C)

Notes

Holes in files are counted as if they actually occupied space.

There is a precedence level to which the options operate regardless of the order they are given on the command line. The **-c** option is first, followed by the **-f** option. For example, “**quot -f -c**” is the same as “**quot -c**”.

See “Notes” under *mount*(C).



Name

`random` – Generates a random number.

Syntax

`random [-s] [ scale ]`

Description

Random generates a random number, *scale*, on the standard output, and returns the number as its exit value. By default, this number is either 0 or 1. If *scale* is given a value between 1 and 255, then the range of the random value is from 0 to *scale*. If *scale* is greater than 255 an error message is printed.

When the `-s` (silent) option is given, then the random number is returned as an exit value but is not printed on the standard output. If an error occurs, *random* returns an exit value of zero.

See Also

`rand(S)`

Notes

This command does not perform any floating point computations.

Random uses the time of day as a seed.



Name

`rcp` – Copies files across XENIX systems.

Syntax

`rcp [ options ] [ srcmachine: ] srcfile [ destmachine: ] destfile`

Description

Rcp copies files between systems in a Micnet network. The command copies the *srcmachine:srcfile* to *destmachine:destfile*, where *srcmachine:* and *destmachine:* are optional names of systems in the network, and *srcfile* and *destfile* are path-names of files. If a machine name is not given, the name of the current system is assumed. If `-` is given in place of *srcfile*, *rcp* uses the standard input as the source. Directories named on the destination machine must be publicly writable, and directories and files named on a remote source machine must be publicly readable.

The available options are:

`-m`

Mails and reports completion of the command, whether there is an error or not.

`-u [machine:]user`

Any mail goes to the named *user* on *machine*. The default *machine* is the machine on which the *rcp* command is completed or on which an error was detected. If an alias for *user* exists in the system alias files on that *machine*, the mail will be redirected to the appropriate mailbox(es). Since system alias files are usually identical throughout the network, any specified *machine* will most likely be overridden by the alias mechanism. To prevent the use of aliases, the *user* must be escaped with at least two `\` characters (at least four if given as a shell command).

Rcp is useful for transferring small numbers of files across the network. The network consists of daemons that periodically awaken and send files from one system to another. The network must be installed using *netutil*(C) before *rcp* can be used. Also, to enable transfer of files from a remote system, the line:

```
rcp=/usr/bin/rcp
```

must be present in the default file, */etc/default/micnet*, on the systems in the network, or:

- 1) "executeall" must be present AND
- 2) the "execpath=PATH=..." must include */usr/bin*

Example

```
rcp -m machine1:/etc/mnttab /tmp/vtape
```

See Also

mail(C), micnet(M), netutil(C), remote(C)

Diagnostics

If an error occurs, mail is sent to the user.

Notes

Full pathnames must be specified for remote files.

Rcp handles binary data files transparently; no extra options or protocols are needed to handle them. Wildcards are not expanded on the remote machine.

Name

`red` – Invokes a restricted version of *ed*(C).

Syntax

`red [ file ]`

Description

Red is a restricted version of *ed*(C). It will only allow editing of files in the current directory. It prohibits executing *sh*(C) commands via the `!` command. *Red* displays an error message on any attempt to bypass these restrictions.

In general, *red* does not allow commands like:

`!date`

or:

`!sh`

Furthermore, *red* will not allow pathnames in its command line. For example, the command:

`red /etc/passwd`

when the current directory is not `/etc`, causes an error.

See Also

ed(C), *rsh*(C).



Name

`remote` – Executes commands on a remote XENIX system.

Syntax

```
remote [ - ] [ -f file ] [ -m ] [ -u user ] machine command [ arguments ]
```

Description

Remote is a limited networking facility that permits execution of XENIX commands across serial lines. Commands on any connected system may be executed from the host system using *remote*. A command line consisting of *command* and any *arguments* separated by blanks is executed on the remote *machine*. The results of the command are mailed to the user on the host machine (unless the `-u` option was used) and are not displayed on the machine on which the command was executed.

A machine's name is located in the `/etc/systemid` file. Note that wildcards are *not* expanded on the remote *machine*, so they should not be specified in *arguments*. The optional `-m` switch causes mail to be sent to the user telling whether the command is successful.

The available options follow:

- A dash signifies that standard input is used as the standard input for *command* on the remote *machine*. Standard input comes from the local host and not from the remote *machine*.
- `-f file` Use the specified *file* as the standard input for *command* on the remote *machine*. The *file* exists on the local host and not on the remote *machine*.
- `-m` Mails the user to report completion of the command. By default, mail reports only errors.
- `-u user` Any mail goes to the named *user* on *machine*. The default *machine* is the machine on which the *rcp* command is completed or on which an error was detected. If an alias for *user* exists in the system alias files on that *machine*, the mail will be redirected to the appropriate mailbox(es). Since system alias files are usually identical throughout the network, any specified *machine* will most likely be overridden by the alias mechanism. To prevent aliases, *user* must be escaped with at least two `\` characters (at least four if given as a shell command).

Before *remote* can be successfully used, a network of systems must first be set up and the proper daemons initialized using *netutil*(C). Also, entries for the command to be executed using *remote* must be added to the `/etc/default/micnet` files on each remote *machine*.

Example

The following command executes an *ls* command on the */tmp* directory of the machine, *machine1*. *Machine1* mails the output of that command to the user on the machine that initiated the *remote* command.

```
remote machine1 ls /tmp
```

See Also

mail(C), micnet(M), netutil(C), rcp(C)

Notes

The *mail* command uses the equivalent of *remote* to send mail between machines.

Name

`restore`, `restor` – Invokes incremental file system restorer.

Syntax

`restore key [ arguments ]`

`restor key [ arguments ]`

Description

Restore is used to read archive media backed up with the *backup(C)* command. The *key* specifies what is to be done. *Key* is one of the characters **rRxt**, optionally combined with **f**. *Restor* is an alternate spelling for the same command.

f Uses the first *argument* as the name of the archive instead of the default.

Fnum Specifies the file number of the first volume to be restored.

kvsiz Specifies the size of the volume to be restored.

r,R The archive is read and loaded into the file system specified in *argument*. This should not be done lightly (see below). If the *key* is **R**, *restore* asks which archive of a multivolume set to start on. This allows *restore* to be interrupted and then restarted (an *fsck* must be done before the restart).

x Each file on the archive named by an *argument* is extracted. The filename has all “mount” prefixes removed; for example, if */usr* is a mounted file system, */usr/bin/lpr* is named */bin/lpr* on the archive. The extracted file is placed in a file with a numeric name supplied by *restore* (actually the inode number). In order to keep the amount of archive read to a minimum, the following procedure is recommended:

1. Mount volume 1 of the set of backup archives.
2. Type the *restore* command.
3. *Restore* will announce whether or not it found the files, give the numeric name that it will assign to the file, and in the case of a tape, rewind to the start of the archive.
4. It then asks you to “mount the desired tape volume”. Type the number of the volume you choose. On a multivolume backup the recommended procedure is to mount the volumes, last through first. *Restore* checks to see if any of the requested files are on the mounted archive (or a later archive—thus the reverse order). If the requested files are not there, *restore* doesn’t read through the tape. If you are working with a single-volume backup or if the number of files being restored is large, respond to the query with **1**, and *restore*

will read the archives in sequential order.

Xfiles Puts files in the directory specified by *arguments*.

t Prints the date the archive was written and the date the file system was backed up.

T This causes *restore* to behave like *dumpdir(C)* except that it doesn't list directories.

The **r** option should only be used to restore a complete backup archive onto a clear file system, or to restore an incremental backup archive onto a file system so created. Thus:

```
/etc/mkfs /dev/hd1 10000
restore r /dev/hd1
```

is a typical sequence to restore a complete backup. Another *restore* can be done to get an incremental backup in on top of this.

A *backup* followed by a *mkfs* and a *restore* is used to change the size of a file system.

Files

*rst** Temporary files

/etc/default/restore Name of default archive device

The default archive unit varies with installation.

Notes

It is not possible to successfully *restore* an entire active root file system.

Diagnostics

There are various diagnostics involved with reading the archive and writing the disk. There are also diagnostics if the i-list or the free list of the file system is not large enough to hold the dump.

If the dump extends over more than one disk or tape, it may ask you to change disks or tapes. Reply with a NEWLINE when the next unit has been mounted.

See Also

backup(C), default(M), fsck(C), mkfs(C)



Name

rewind – Rewinds the tape cartridge drive.

 Syntax

rewind

Description

Rewind rewinds the tape drive, if a tape is mounted on it. This is done by opening and closing the tape drive, using a file name that rewinds when the file is closed.

See Also

mt(M), skip(C)

Files

/dev/rmt0





Name

`rm`, `rmdir` – Removes files or directories.

Syntax

`rm [ -fri ] file ...`

`rmdir dir ...`

Description

Rm removes the entries for one or more files from a directory. If an entry was the last link to the file, the file is destroyed. Removal of a file requires write permission in its directory, but it does not require read or write permission on the file itself.

If a file has no write permission and the standard input is a terminal, its permissions are printed and a line is read from the standard input. If that line begins with “y”, the file is deleted, otherwise the file remains. No questions are asked when the `-f` option is given, if the standard input is not a terminal, or if you are logged in as super-user.

If a designated file is a directory, an error comment is printed unless the optional argument `-r` has been used. In that case, *rm* recursively deletes the entire contents of the specified directory, and the directory itself.

If the `-i` (interactive) option is in effect, *rm* asks whether to delete each file, and, if the `-r` option is in effect, whether to examine each directory.

Rmdir removes empty directories.

Diagnostics

It is forbidden to remove the `..` file to avoid the consequences of inadvertently doing something like:

```
rm -r .*
```

It is also forbidden to remove the root directory of a given file system.

No more than 17 levels of subdirectories can be removed using the `-r` option.



Name

rmuser – Removes a user from the system.

Syntax

/etc/rmuser

Description

Rmuser removes users from the system. It begins by prompting for a user name; after receiving a valid user name as a response, it then deletes the named user's entry in the password file, removes all the files in the user's home directory, and the home directory itself. It will also remove the users group entry in **/etc/group** if the user was the only remaining member of that group, and if the group ID was greater than 50.

Before removing a user ID from the system, make sure its mailbox is empty and that all files belonging to that user ID have been saved or deleted as required.

The *rmuser* program will refuse to remove a user ID or any of its files if one or more of the following checks fails:

- The user name given is one of the “system” user names such as **root**, **sys**, **sysinfo**, **cron**, or **uucp**. All user IDs less than 200 are considered reserved for system use, and cannot be removed using *rmuser*. Likewise, all group IDs less than 50 are not removable using *rmuser*.
- The user's mailbox exists and is not empty.
- The user's home directory contains files other than **.profile**, **.login**, **.cshrc**, **.mailrc**, **.exrc**, **.newsrsc**, **.plan**, and **.project**.

Rmuser can be used only by the super-user.

Files

/etc/passwd

/usr/spool/mail/username

\$HOME

RMUSER (C)

RMUSER (C)

See Also

backup(C), mkuser(C)

Name

`rsh` – Invokes a restricted shell (command interpreter).

Syntax

```
rsh [flags] [name [arg1 ... ]]
```

Description

Rsh is a restricted version of the standard command interpreter *sh*(C). It is used to set up login names and execution environments whose capabilities are more controlled than those of the standard shell. The actions of *rsh* are identical to those of *sh*, except that changing directory with *cd*, setting the value of `$PATH`, using command names containing slashes, and redirecting output using `>` and `>>` are all disallowed.

When invoked with the `-rsh` name, *rsh* reads the user's `.profile` (from `$HOME/.profile`). It acts as the standard *sh* while doing this, except that an interrupt causes an immediate exit instead of causing a return to command level. The restrictions above are enforced after `.profile` is interpreted.

When a command to be executed is a shell procedure, *rsh* invokes *sh* to execute it. Thus, it is possible to provide the end-user with shell procedures that have access to the full power of the standard shell, while restricting this user to a limited menu of commands; this scheme assumes that the end-user does not have write and execute permissions in the same directory.

The net effect of these rules is that the writer of the `.profile` has complete control over user actions, first, by performing guaranteed setup actions, and second, by leaving the user in an appropriate directory (probably *not* the login directory).

Rsh is actually just a link to *sh* and any *flags* arguments are the same as for *sh*(C).

The system administrator often sets up a directory of commands that can be safely invoked by *rsh*.

See Also

`profile`(M), `sh`(C)



Name

runbig - Runs a command that may require more memory than the default allows.

Syntax

runbig *command* [*arguments*]

Description

Runbig executes *command* with more memory than is normally available to a user process. When *runbig* executes the specified *command*, it removes the restriction on the default size of memory available to the user process. The *command* will run normally unless it is, or grows to be, larger than the maximum amount of memory available to a user process. It is then locked in core memory and not swapped until it either exits or shrinks to a size less than or equal to the maximum size of a user process.

The removal of the process size restriction during execution of *command* will be preserved on an *exec(S)* system call, but not on a *fork(S)* system call.

See Also

exec(S), *fork(S)*, *proct(S)*

Notes

Running programs which are greater than the default maximum process size, and which are possibly greater than the size of the disk swap area, may severely impact system performance.

Runbig has no effect on systems whose memory size is much less than the size of the disk swap area.



Name

sddate – Prints and sets backup dates.

Syntax

sddate [*name lev date*]

Description

If no argument is given, the contents of the backup date file, */etc/ddate*, are printed. The backup date file is maintained by *backup(C)* and contains the date of the most recent backup for the backup level of each file system.

If arguments are given, an entry is replaced or made in */etc/ddate*. *Name* is the last component of the device pathname. *Lev* is the backup level number (from 0 to 9), and *date* is a time in the form taken by *date(C)*:

mmddhhmm[*yy*]

Where the first *mm* is a two-digit month in the range 01-12, *dd* is a two-digit day of the month, *hh* is a two-digit military hour from 00-23, and the final *mm* is a two-digit minute from 00-59. An optional two-digit year, *yy*, is presumed to be an offset from the year 1900, i.e., 19yy.

Some sites may wish to back up file systems by copying them to backup media. *Sddate* could be used to make a “level-0” entry in */etc/ddate*, which would then allow incremental backups. For example:

```
sddate rhd0 5 10081520
```

makes an */etc/ddate* entry showing a level 5 backup of */dev/rhd0* on October 8, at 3:20 PM.

Files

/etc/ddate

See Also

backup(C), date(C)

Diagnostics

bad conversion If the date set is incorrect syntactically.

Name

`sdiff` – Compares files side-by-side.

Syntax

`sdiff [ options ... ] file1 file2`

Description

Sdiff uses the output of *diff*(C) to produce a side-by-side listing of two files indicating the lines that are different. The lines of the two files are printed with a blank gutter between them if they are identical; a < is put in the gutter if the line only exists in *file1*, a > is put in the gutter if the line only exists in *file2*, and a | is put in the gutter for lines that are different.

For example:

x		y
a		a
b	<	
c	<	
d		d
	>	c

The following options exist:

- w *n*** Uses the next argument, *n*, as the width of the output line. The default line length is 130 characters.
- l** Only prints the left side of any lines that are identical.
- s** Does not print identical lines.
- o *output*** Uses the next argument, *output*, as the name of a third file created by merging *file1* with *file2*. Identical lines of *file1* and *file2* are copied to *output*. Sets of differences, as produced by *diff*(C), are printed; sets of differences share a common gutter character. After printing each set of differences, *sdiff* prompts the user with a % and waits for you to type one of the following commands:

- l** Appends the left column to the output file
- r** Appends the right column to the output file
- s** Turns on silent mode; does not print identical lines

- v Turns off silent mode
- e l Calls the editor with the left column
- e r Calls the editor with the right column
- e b Calls the editor with the concatenation of left and right
- e Calls the editor with a zero length file
- q Exits from the program

Upon exiting from the editor, the resulting file is concatenated onto the end of the *output* file.

See Also

diff(C), ed(C)

Name

`sed` – Invokes the stream editor.

Syntax

```
sed [ -n ] [ -e script ] [ -f sfile ] [ files ]
```

Description

Sed copies the named *files* (standard input default) to the standard output, edited according to a script of commands. The `-f` option causes the script to be taken from *sfile*; these options accumulate. If there is just one `-e` option and no `-f` options, the flag `-e` may be omitted. The `-n` option suppresses the default output. A script consists of editing commands, one per line, in the following form:

```
[ address [ , address ] ] function [ arguments ]
```

In normal operation, *sed* cyclically copies a line of input into a *pattern space* (unless there is something left after a *D* command), applies in sequence all commands whose *addresses* select that pattern space, and at the end of the script copies the pattern space to the standard output (except under `-n`) and deletes the pattern space.

Some of the commands use a *hold space* to save all or part of the pattern space for subsequent retrieval.

An *address* is either a decimal number that counts input lines cumulatively across files, a `$` that addresses the last line of input, or a context address, i.e., a *regular expression* in the style of *ed(C)* modified as follows:

- In a context address, the construction `\?regular expression\?`, where `?` is any character, is identical to `/regular expression/`. Note that in the context address, `\xabc\xdefx`, the second `x` stands for itself, so that the regular expression is `abcxdef`.
- The escape sequence, `\n`, matches a NEWLINE *embedded* in the pattern space.
- A period (`.`) matches any character except the *terminal* NEWLINE of the pattern space.
- A command line with no addresses selects every pattern space.
- A command line with one address selects each pattern space that matches the address.

- A command line with two addresses selects the inclusive range from the first pattern space that matches the first address through the next pattern space that matches the second. (If the second address is a number less than or equal to the line number first selected, only one line is selected.) Thereafter, the process is repeated, looking again for the first address.

Editing commands can be applied only to unselected pattern spaces by use of the negation function !.

In the following list of functions the maximum number of permissible addresses for each function is indicated in parentheses.

The *text* argument consists of one or more lines, all but the last of which end with backslashes to hide the NEWLINES. Backslashes in text are treated like backslashes in the replacement string of an *s* command, and may be used to protect initial spaces and TABs against the stripping that is done on every script line. The *rfile* or *wfile* argument must terminate the command line and must be preceded by exactly one space. Each *wfile* is created before processing begins. You can use at most ten distinct *wfile* arguments.

- (1) **a**
text Appends *text*, placing it on the output before reading the next input line.
- (2) **b label** Branches to the colon (:) command bearing the *label*. If *label* is empty, branches to the end of the script.
- (2) **c**
text Changes text by deleting the pattern space and then appending *text*. With a 0 or 1 address or at the end of a 2-address range, it places *text* on the output and starts the next cycle.
- (2) **d** Deletes the pattern space and starts the next cycle.
- (2) **D** Deletes the initial segment of the pattern space through the first NEWLINE and starts the next cycle.
- (2) **g** Replaces the contents of the pattern space with the contents of the hold space.
- (2) **G** Appends the contents of the hold space to the pattern space.
- (2) **h** Replaces the contents of the hold space with the contents of the pattern space.

- (2) **H** Appends the contents of the pattern space to the hold space.
- (1) **i**
text Insert. Places *text* on the standard output.
- (2) **l** Lists the pattern space on the standard output with nonprinting characters spelled in two-digit ASCII with long lines folded.
- (2) **n** Copies the pattern space to the standard output. Replaces the pattern space with the next line of input.
- (2) **N** Appends the next line of input to the pattern space with an embedded NEWLINE. (The current line number changes.)
- (2) **p** Prints (copies) the pattern space on the standard output.
- (2) **P** Prints (copies) the initial segment of the pattern space through the first NEWLINE to the standard output.
- (1) **q** Quits *sed* by branching to the end of the script. No new cycle is started.
- (2) **r** *rfile* Reads the contents of *rfile* and places them on the output before reading the next input line.
- (2) **s**/*regular expression/replacement/flags*
Substitutes the *replacement* string for instances of the *regular expression* in the pattern space. Any character may be used instead of */*. For a more detailed description see *ed(C)*. *Flags* is zero or more of the following:
- g** Globally substitutes for all nonoverlapping instances of the *regular expression* rather than just the first one.
 - p** Prints the pattern space if a replacement was made.
 - w** *wfile*
Writes the pattern space to *wfile* if a replacement was made.
- (2) **t** *label* Branches to the colon (:) command bearing *label* if any substitutions have been made since the most recent reading of an input line or execution of a *t* command. If *label* is empty, *t* branches to the end of the script.
- (2) **w** *wfile* Writes the pattern space to *wfile*.
- (2) **x** Exchanges the contents of the pattern and hold spaces.

(2) *y/string1/string2/*

Replaces all occurrences of characters in *string1* with the corresponding characters in *string2*. The lengths of *string1* and *string2* must be equal.

(2) *!function*

Applies the *function* (or group, if *function* is {}) only to lines *not* selected by the address(es).

(0) *:label* This command does nothing; it bears a *label* for *b* and *t* commands to branch to.

(1) *=* Places the current line number on the standard output as a line.

(2) *{* Executes the following commands through a matching *}* only when the pattern space is selected.

(0) An empty command is ignored.

See Also

awk(C), ed(C), grep(C)

XENIX Text Processing Guide

Notes

This command is more fully documented in the *XENIX Text Processing Guide*.

Name

setclock – Sets/displays the real-time clock.

Syntax

setclock [*time*]

Description

Setclock sets the battery-powered, real-time clock of your computer to the specified *time*. If you omit *time*, *setclock* displays the current time from the real-time clock.

Specify the time in this form:

MMddhhmm[yy]

where: MM is the month

dd is the day

hh is the hour (24-hour time)

mm is the minute

yy is the last 2 digits of the year

If you omit *yy*, *setclock* uses the current year as found in the current system time.

Example

To set the real-time clock to 3:33pm on March 19, 1985, type this command:

```
setclock 0319153385
```

and press RETURN.

File

/bin/setclock



Name

`setcolor` – Sets foreground and background colors on a color monitor.

Syntax

`setcolor options [fcolor bcolor]`

Description

Use the `setcolor` command to change the foreground and background colors on the color monitor. *Fcolor* specifies the foreground color, and *bcolor* specifies the optional background color. You can select one of the following colors for background and foreground:

black red green orange
blue cyan white magenta

In addition, you can select an intensified color for the foreground, thus increasing the number of colors to 16.

If you specify `setcolor` without any options, XENIX displays its usage message and the list of available colors.

The following options are available:

- `-n` Sets the screen to normal. This is white characters on black background.
- `-b color` Sets the background color to the specified *color*.
- `-i` Sets the foreground to “intense.”

Files

`/bin/setcolor`

Notes

Do not set foreground and background to the same color. If you do, you won’t be able to see what you are typing or what is displayed. If you should accidentally do this, carefully type:

```
setcolor -n
```

and press RETURN.



Name

setkey – Assigns values to the function keys.

Syntax

```
setkey key 'string'
setkey key -d [keyvalue]
```

Description

Setkey assigns the specified string to the function key (or key combination) represented by *key*. You can program as many as 40 key combinations. *Key* can be one of the following:

```
function-key  assigns string to function-key (1-10).
s function-key assigns the string to SHIFT-function-key.
c function-key assigns the string to CTRL-function-key.
a function-key assigns the string to ALT-function-key.
```

String can contain control codes. For example, pressing RETURN within the string, so the command executes automatically. When including control characters, enclose *string* in quotation marks so it is not processed by the shell. If you are using *csh*, precede the control code with a space and a backslash (\).

Use the *-d* option to assign a predefined key value to a function key.

Examples

To set function key 2 to automatically execute the commands *pwd* and *lc*, type:

```
setkey 2 "pwd ; lc <RETURN>"
```

This command would work only in *sh*. If you are using *csh*, use this command:

```
setkey 2 ``pwd ; lc <RETURN>'' <RETURN>
```

There is another way to assign one of the ASCII control characters (0x00 0x20) to a function key. The *string* may be set to: "*^(Hex character + 0x20)*." For example, to assign a /s-1CTRL-J code to function key 1, the following command line can be used:

```
setkey 1 '^*'
```

Notice that 0x0A (RETURN) plus 0x20 (*).

Using the **-d** option, you can assign a predefined key value to a function key. For example:

```
setkey 1 -d 254
```

assigns the 8-bit character code associated with the decimal value 254 (Hex FE) to function key 1.

Files

/bin/setkey

Notes

There can be no spaces between the option (s, c, or a) and the function key number.

Name

setmnt – Establishes `/etc/mnttab` table.

Syntax

`/etc/setmnt`

Description

Setmnt creates the `/etc/mnttab` table (see *mnttab*(F)), which is needed for both the *mount*(C) and *umount*(C) commands. *Setmnt* reads the standard input and creates a *mnttab* entry for each line. Input lines have the format:

filesystem node

where *filesystem* is the name of the file system's *special file* (e.g., `"hd0"`) and *node* is the root name of that file system. Thus, *filesystem* and *node* become the first two strings in the *mnttab*(F) entry.

Files

`/etc/mnttab`

See Also

mnttab(F)

Notes

Errors can occur if *filesystem* or *node* are longer than 128 characters.

Setmnt silently enforces an upper limit on the maximum number of *mnttab* entries.

Setmnt is normally invoked by `/etc/rc` when the system boots up.



Name

`settime` – Changes the access and modification dates of files.

Syntax

`settime mmddhhmm [ yy ] [ -f fname ] name ...`

Description

Sets the access and modification dates for one or more files. The dates are set to the specified date, or to the access and modification dates of the file specified via `-f`. Exactly one of these methods must be used to specify the new date(s). The first *mm* is the month number; *dd* is the day number in the month; *hh* is the hour number (24 hour system); the second *mm* is the minute number; *yy* is the last two digits of the year and is optional. For example:

`settime 1008004583 ralph pete`

sets the access and modification dates of files *ralph* and *pete* to Oct 8, 12:45 AM, 1983. Another example:

`settime -f ralph john`

This sets the access and modification dates of the file, *john*, to those of the file, *ralph*.

Notes

Use of *touch* in place of *settime* is encouraged.



Name

sh – Invokes the shell command interpreter.

Syntax

sh [*-ceiknrstuvx*] [*args*]

Description

The shell is the standard command programming language that executes commands read from a terminal or a file. See *Invocation* below for the meaning of arguments to the shell.

Commands

A *simple-command* is a sequence of *words* separated by *blanks* (a *blank* is a TAB or a SPACEBAR). The first word specifies the name of the command to be executed. Except as specified below, the remaining words are passed as arguments to the invoked command. The command name is passed as argument 0 (see *exec(S)*). The *value* of a simple-command is its exit status if it terminates normally, or (octal) 1000+*status* if it terminates abnormally, i.e., if the failure produces a *corefile*. See *signal(S)* for a list of status values.

A *pipeline* is a sequence of one or more *commands* separated by a vertical bar (|). (The caret (^) also has the same effect.) The standard output of each command but the last is connected by a *pipe(S)* to the standard input of the next command. Each command is run as a separate process; the shell waits for the last command to terminate.

A *list* is a sequence of one or more pipelines separated by ;, &, &&, or ||, and optionally terminated by ; or &. Of these four symbols, ; and & have equal precedence, which is lower than that of && and ||. The && and || symbols also have equal precedence. A semicolon (;) causes sequential execution of the preceding pipeline; an ampersand (&) causes asynchronous execution of the preceding pipeline (i.e., the shell does *not* wait for that pipeline to finish). The && (||) symbol causes the *list* following it to be executed only if the preceding pipeline returns a zero (nonzero) exit status. An arbitrary number of NEWLINES may appear in a *list*, instead of semicolons, to delimit commands.

A *command* is either a simple-command or one of the following commands. Unless otherwise stated, the value returned by a ccommand is that of the last simple-command executed in the command:

for *name* [*in word* . . .] do *list* done

Each time a *for* command is executed, *name* is set to the next *word* taken from the *in word* list. If *in word* is omitted, then the *for* command executes the *do list* once for each positional parameter that is set (see *Parameter Substitution*). Execution ends when there are no more words in the list.

case word in [*pattern* [|*pattern*] ...] *list* ;;] ... esac

A *case* command executes the *list* associated with the first *pattern* that matches *word*. The form of the patterns is the same as that used for filename generation (see *Filename Generation*).

if *list* then *list* [elif *list* then *list*] ... [else *list*] fi

The *list* following *if* is executed and, if it returns a zero exit status, the *list* following the first *then* is executed. Otherwise, the *list* following *elif* is executed and, if its value is zero, the *list* following the next *then* is executed. Failing that, the *else list* is executed. If no *else list* or *then list* is executed, the *if* command returns a zero exit status.

while *list* do *list* done

A *while* command repeatedly executes the *while list* and, if the exit status of the last command in the *list* is zero, executes the *do list*; otherwise the loop terminates. If no commands in the *do list* are executed, the *while* command returns a zero exit status; *until* may be used in place of *while* to negate the loop termination test.

(*list*)

Executes *list* in a subshell.

{*list*;}

list is simply executed.

The following words are recognized only as the first word of a command and when not quoted:

if then else elif fi case esac for while until do done { }

Comments

A word beginning with # causes that word and all the following characters up to a NEWLINE to be ignored.

Command Substitution

The standard output from a command enclosed in a pair of grave accents (``) may be used as part or all of a word; trailing NEWLINES are removed.

Parameter Substitution

The \$ character is used to introduce substitutable *parameters*. Positional parameters may be assigned values by *set*. Variables may be set by writing: